



Introdução à Engenharia de Software

Bem-vindos à disciplina de Engenharia de Software! Neste semestre, exploraremos os fundamentos, métodos e práticas essenciais para o desenvolvimento de software de qualidade. Esta apresentação introduz nossa jornada de aprendizagem e como trabalharemos juntos para maximizar seu desenvolvimento profissional.

Sobre a Disciplina

Objetivos de Aprendizagem

- Compreender os fundamentos da Engenharia de Software
- Dominar técnicas de elicitação e análise de requisitos
- Conhecer diferentes modelos de ciclo de vida
- Aplicar metodologias ágeis em projetos práticos
- Desenvolver habilidades analíticas e críticas



Metodologia de Avaliação

Atividade Avaliativa (A1)

Teste com questões teóricas e práticas sobre os temas abordados nas primeiras aulas.

Data: 24/09 (7º encontro)

Artigo Individual

Desenvolvimento de um artigo científico sobre tema relacionado à Engenharia de Software. Acompanhamento nas aulas complementares.

Avaliação Final (A2)

Prova abrangendo todo o conteúdo do semestre.

Data: 12/11 (14º encontro)

Uso de Inteligência Artificial



O papel da IA no desenvolvimento moderno de software. Nesta disciplina:

- O uso de ferramentas de IA é **permitido e incentivado**
- Toda contribuição da IA deve ser **validada** pelo aluno
- O professor auxiliará na verificação do conteúdo gerado
- A compreensão do material continua sendo responsabilidade do aluno

Oportunidades Complementares



Cursos de Git

Participação em cursos de controle de versão usando Git pode contar como atividade complementar para avaliação.



Linguagens de Programação

Cursos livres de linguagens relevantes para Engenharia de Software serão considerados para complementação de nota.

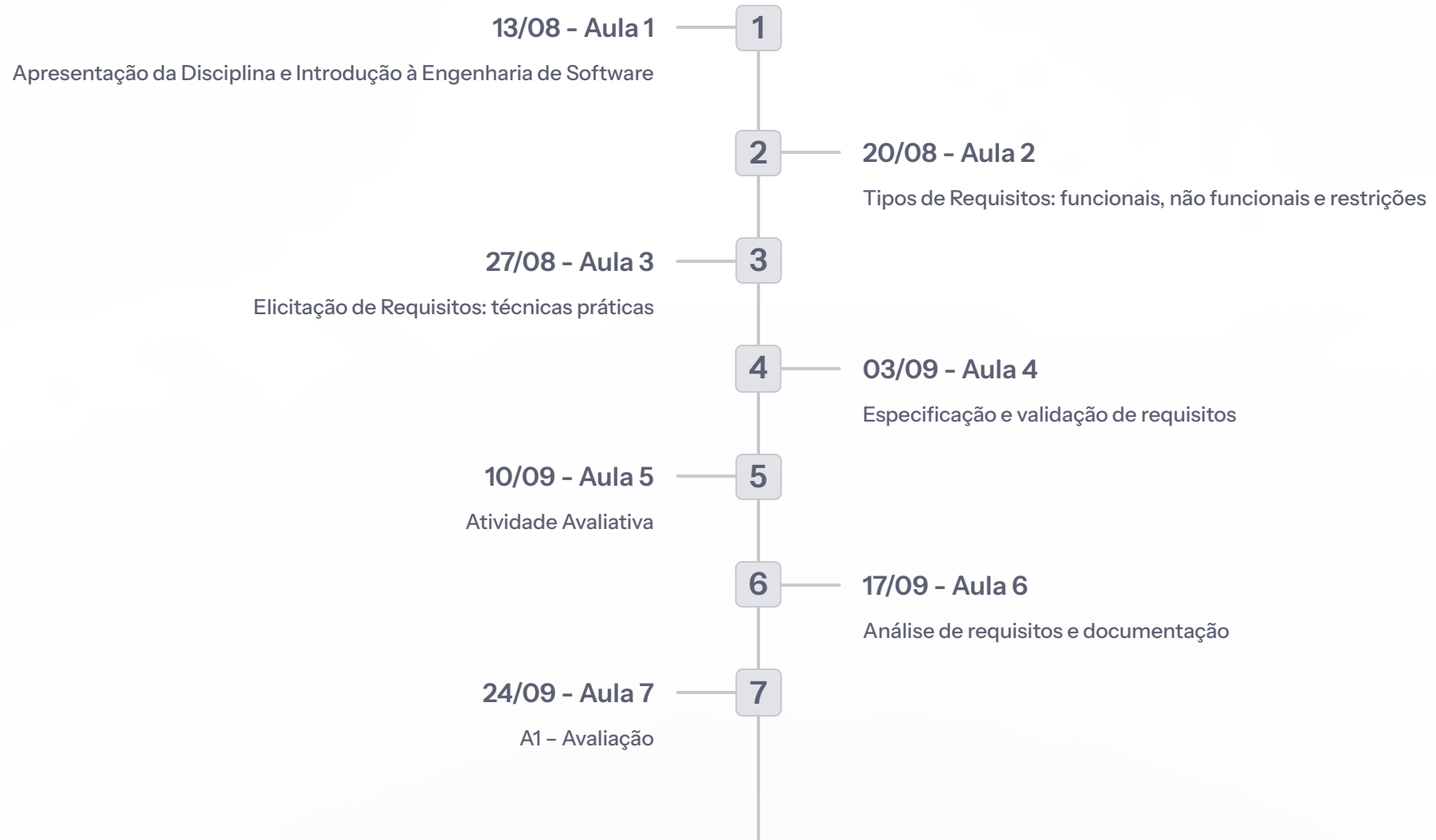


Certificações

Certificações na área de Engenharia de Software obtidas durante o semestre podem ser integradas à avaliação.

Todas as atividades complementares devem ser apresentadas ao professor para validação e incorporação na avaliação final.

Conteúdo Programático (Parte 1)



Conteúdo Programático (Parte 2)



Sobre o Artigo Individual

Diretrizes para o Desenvolvimento

- Tema relacionado à área de Engenharia de Software
- Formato acadêmico com introdução, desenvolvimento, conclusão e referências

O artigo deve demonstrar capacidade de pesquisa, análise crítica e síntese de conhecimentos adquiridos durante a disciplina.



Definição de Engenharia de Software

A Engenharia de Software é uma disciplina fundamental no cenário tecnológico atual, que vai muito além da simples escrita de código. Ela se concentra em aplicar princípios de engenharia para o desenvolvimento de software.

Segundo a renomada definição de Ian Sommerville, um dos maiores nomes da área:

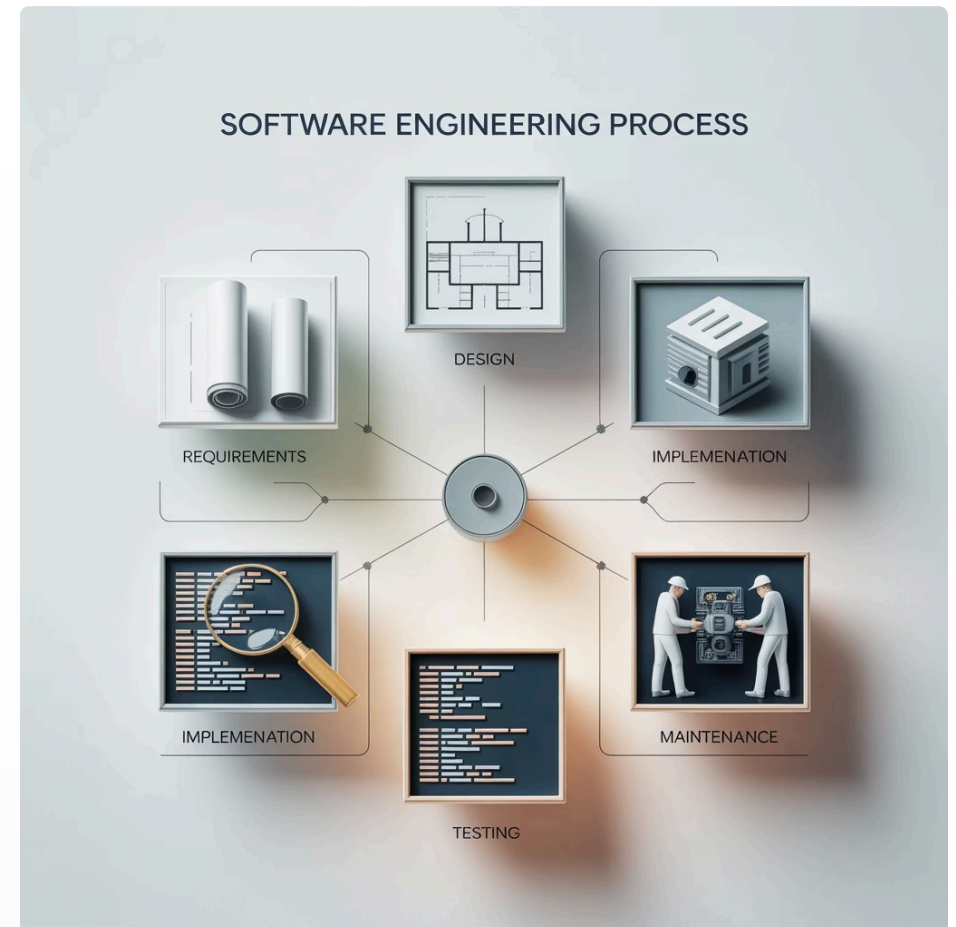
“Engenharia de Software é a aplicação de uma abordagem sistemática, disciplinada e quantificável ao desenvolvimento, operação e manutenção de software.”

Essa definição ressalta a diferença crucial entre a engenharia de software e a programação isolada. Enquanto a programação foca na implementação de funcionalidades específicas, a engenharia de software abrange todo o ciclo de vida do produto.

Isso significa que ela envolve uma série de aspectos interligados:

- **Processos:** Definição de metodologias e fluxos de trabalho estruturados para gerenciar o desenvolvimento (ex: Agile, Waterfall).
- **Ferramentas:** Utilização de softwares e plataformas que auxiliam em todas as fases, desde o planejamento até a entrega (ex: IDEs, sistemas de controle de versão, ferramentas de gerenciamento de projetos).
- **Técnicas:** Aplicação de métodos e práticas para garantir a qualidade, eficiência e manutenibilidade do código (ex: padrões de projeto, refatoração, testes automatizados).

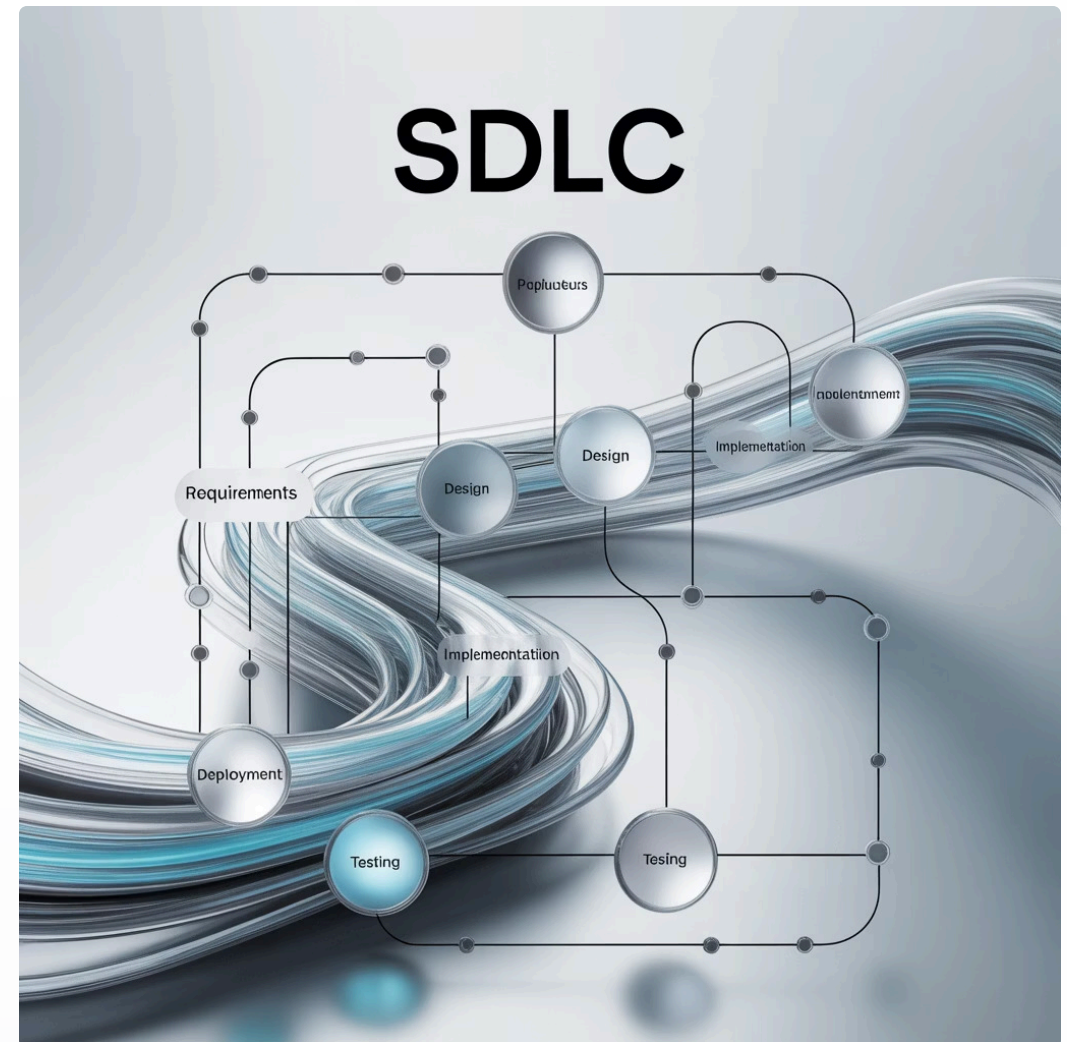
A adoção dessa abordagem sistemática e disciplinada é essencial para criar software robusto, escalável e de alta qualidade que atenda às necessidades dos usuários de forma consistente e eficiente.



Ciclo de Vida do Software (SDLC)

O Ciclo de Vida do Desenvolvimento de Software (SDLC - Software Development Life Cycle) é um conjunto de fases estruturadas que descreve o processo de desenvolvimento, entrega e manutenção de software, desde a concepção inicial até a sua desativação. Ele serve como um roteiro, garantindo que todas as etapas necessárias sejam cumpridas para produzir um software de alta qualidade, que atenda às necessidades do usuário e seja entregue dentro do prazo e orçamento.

1. **Levantamento de Requisitos** Esta fase inicial é crucial para entender o que o cliente realmente precisa. Envolve a coleta, análise e documentação detalhada das funcionalidades esperadas do software, das restrições e dos objetivos do negócio. É a base para todas as etapas subsequentes e busca evitar retrabalhos no futuro.
2. **Projeto / Design** Nesta etapa, os requisitos coletados são transformados em um plano técnico. Define-se a arquitetura do software, o design da interface do usuário (UI/UX), a estrutura do banco de dados e a integração com outros sistemas. O objetivo é criar um projeto detalhado que guiará a equipe de desenvolvimento.
3. **Implementação** É a fase onde o código é escrito, com base nos documentos de design. Os desenvolvedores transformam as especificações em software funcional, seguindo as melhores práticas de codificação e utilizando ferramentas de controle de versão para gerenciar o processo.
4. **Testes** Após a implementação, o software passa por rigorosos testes para identificar e corrigir defeitos. Isso inclui testes de unidade, integração, sistema e aceitação, garantindo que o software funcione conforme o esperado e atenda aos requisitos de qualidade e desempenho.
5. **Implantação / Operação** Uma vez testado e validado, o software é liberado para uso. Esta fase envolve a instalação do sistema no ambiente do cliente, configuração, treinamento de usuários e o início das operações. É o momento em que o software se torna acessível aos seus usuários finais.
6. **Manutenção** Mesmo após a implantação, o software requer manutenção contínua. Isso inclui a correção de bugs que podem surgir, aprimoramento de desempenho, adaptação a novas tecnologias e a adição de novas funcionalidades para garantir que o sistema permaneça relevante e eficiente ao longo do tempo.



Cada fase do SDLC é interconectada e iterativa em muitos modelos modernos, permitindo flexibilidade e adaptação às mudanças. Compreender o SDLC é fundamental para qualquer profissional da área de Engenharia de Software, pois ele fornece uma estrutura para gerenciar a complexidade do desenvolvimento de sistemas.

Princípios Fundamentais da Engenharia de Software

A Engenharia de Software se baseia em um conjunto de princípios essenciais que guiam o desenvolvimento, garantindo a construção de sistemas eficazes, eficientes e de alta qualidade. Compreender e aplicar esses pilares é crucial para o sucesso de qualquer projeto de software.



Qualidade

A busca pela qualidade é central na Engenharia de Software. Isso significa garantir que o software seja confiável, seguro, eficiente e fácil de manter ao longo do tempo. Inclui aspectos como a ausência de defeitos (bug-free), performance adequada, segurança contra vulnerabilidades e a capacidade de ser facilmente adaptado e evoluído para novas necessidades.



Processos

A padronização e estruturação das etapas de desenvolvimento são fundamentais. A definição de processos claros e repetíveis permite que as equipes trabalhem de forma consistente, reduzindo a variabilidade e aumentando a previsibilidade dos resultados. Um processo bem definido abrange desde a concepção e requisitos até a implantação e manutenção do software.

A integração desses princípios permite que engenheiros de software construam soluções robustas e sustentáveis, alinhadas às expectativas dos usuários e às demandas do mercado.



Gerenciamento de Projetos

O gerenciamento eficaz de projetos é vital para entregar software dentro do prazo e orçamento. Este princípio abrange o planejamento cuidadoso, a alocação de recursos, a gestão de riscos e a coordenação de equipes. Ferramentas e metodologias (como Agile ou Waterfall, dependendo do contexto) são aplicadas para monitorar o progresso e garantir que o projeto atinja seus objetivos.



Ferramentas e Técnicas

A aplicação de ferramentas e técnicas adequadas otimiza todo o ciclo de vida do software. Isso inclui a automação de tarefas repetitivas (como testes e implantação contínua), a utilização de métricas para avaliar o desempenho e a qualidade, a aplicação de técnicas de design e codificação, e o uso de sistemas de controle de versão (como Git) para gerenciar o código de forma colaborativa e segura.

Objetivos da Engenharia de Software

A Engenharia de Software possui metas bem definidas que visam otimizar todo o processo de desenvolvimento, garantindo resultados superiores e sustentáveis. Estes objetivos são fundamentais para o sucesso e a longevidade de qualquer projeto de software.

- **Entregar software funcional e de qualidade no prazo e orçamento:** Este é o objetivo primordial. Visa assegurar que o software não apenas execute suas funções esperadas corretamente, mas que também seja confiável, eficiente, seguro e fácil de usar. Cumprir o cronograma e o orçamento estabelecidos é crucial para a viabilidade do projeto e para atender às expectativas dos stakeholders.
- **Reduzir riscos de falhas e retrabalho:** Através de metodologias e práticas sistemáticas, a Engenharia de Software busca minimizar a ocorrência de bugs, vulnerabilidades de segurança e erros de arquitetura. Isso evita retrabalhos caros e demorados, que consomem recursos e atrasam a entrega, otimizando a eficiência do desenvolvimento.
- **Garantir manutenibilidade e evolução do sistema:** Um software de qualidade deve ser fácil de manter, corrigir e adaptar ao longo do tempo. Este objetivo foca na criação de código limpo, modular e bem documentado, permitindo que futuras equipes possam entender, modificar e estender o sistema de forma eficiente para atender a novas demandas e tecnologias.
- **Fornecer documentação e métricas para melhorar processos futuros:** A criação de documentação clara (requisitos, design, manuais de usuário) e a coleta de métricas de projeto (desempenho, defeitos, tempo de desenvolvimento) são essenciais. Esses dados servem como base para a análise e melhoria contínua dos processos de desenvolvimento, promovendo um ciclo de aprendizado e aprimoramento em projetos subsequentes.



BIBLIOGRAFIA BÁSICA:

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: uma abordagem profissional. McGraw-Hill, 2016.

SOMMERVILLE, Ian. Engenharia de Software. 9. ed. São Paulo:Pearson, 2011.

REINEHR, Sheila. Engenharia de requisitos. Porto Alegre: SAGAH, 2020.